Data Analytics Project: House Sales Analysis

Submitted By: Team1

1 Bharath
2 Akash
3 Nagavenkatabhaskar
4 Amulya
5 Sri Lekha

Dataset Used: kc_house_data.csv

This project analyzes house sales data to understand how different factors affect house prices. The dataset includes details like price, size, number of bedrooms, and location. Using data cleaning, analysis, and visualization, we find useful patterns and trends. The main goal is to identify what influences house prices the most.

Project Goal: The goal of this project is to analyze house sales data and understand the factors that affect house prices. It aims to identify important features such as size, location, and number of rooms that influence pricing. The project also helps in finding patterns and trends in the data using analysis and visualization techniques.

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (np.diff).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts)

🎯 **Project Coverage**

Category	Skills Covered
Data Preprocessing	Data Loading, Missing Value Handling (Mean/Mode), Data Cleaning, Type Conversion.
Pandas Operations	Column Selection, Filtering, Grouping, Aggregation, Value Count, Feature Engineering (Height Category creation)
NumPy Calculations	Array conversion, Numerical analysis using (np.diff) for height differences
Visualization	Trend Analysis, Line Chart, Bar Chart, Pie Chart, Histogram, Stacked Bar Chart, Saving Plots

Scenario 1: Data Loading & Basic Cleaning

Description:
In this scenario,
The House Sales dataset (kc_house_data.csv) is loaded using the Pandas library.
The first 5 rows and column names are displayed to understand the dataset structure.
Missing values are checked in important columns like bedrooms, bathrooms, sqft_living, and price.
Missing values are filled using mode for bedrooms and mean for bathrooms, sqft_living, and price.
Finally, the columns are converted to numeric format and checked again to ensure the data is clean.

Task	Description
1.	Load the dataset using Pandas.
2.	Display: ○ First 5 rows ○ Column names
3.	Check for missing values in: ○ bedrooms ○ bathrooms ○ sqft_living ○ price
4.	ill missing values: ○ bedrooms → use mode ○ bathrooms → use mean ○ sqft_living → use mean ○ price → use mean
5.	Convert these columns to numeric if required: ○ bedrooms ○ bathrooms ○ sqft_living ○ price

Results:

```
First 5 Rows:
           id            date     price  bedrooms  bathrooms  sqft_living  ...  yr_renovated  zipcode      lat     long  sqft_living15  sqft_lot15
0  7129300520  20141013T000000  221900.0         3       1.00         1180  ...             0    98178  47.5112 -122.257           1340        5650
1  6414100192  20141209T000000  538000.0         3       2.25         2570  ...          1991    98125  47.7210 -122.319           1690        7639
2  5631500400  20150225T000000  180000.0         2       1.00          770  ...             0    98028  47.7379 -122.233           2720        8062
3  2487200875  20141209T000000  604000.0         4       3.00         1960  ...             0    98136  47.5208 -122.393           1360        5000
4  1954400510  20150218T000000  510000.0         3       2.00         1680  ...             0    98074  47.6168 -122.045           1800        7503

[5 rows x 21 columns]

Column Names:
Index(['id', 'date', 'price', 'bedrooms', 'bathrooms', 'sqft_living',
       'sqft_lot', 'floors', 'waterfront', 'view', 'condition', 'grade',
       'sqft_above', 'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode',
       'lat', 'long', 'sqft_living15', 'sqft_lot15'],
      dtype='str')

Missing Values Before Filling:
bedrooms: 0
bathrooms: 0
sqft_living: 0
price: 0
```

Conclusion:

This scenario helps in preparing the dataset for further analysis by performing basic cleaning operations. Missing values were identified and replaced using suitable statistical methods like mode and mean. Converting columns into numeric format improves data accuracy and prevents errors in future calculations.

As a result, the dataset becomes clean, consistent, and ready for advanced data analysis such as price prediction, visualization, and trend analysis in house sales.

Scenario 2: Line Graph (Price Trend) (🟢 Easy)

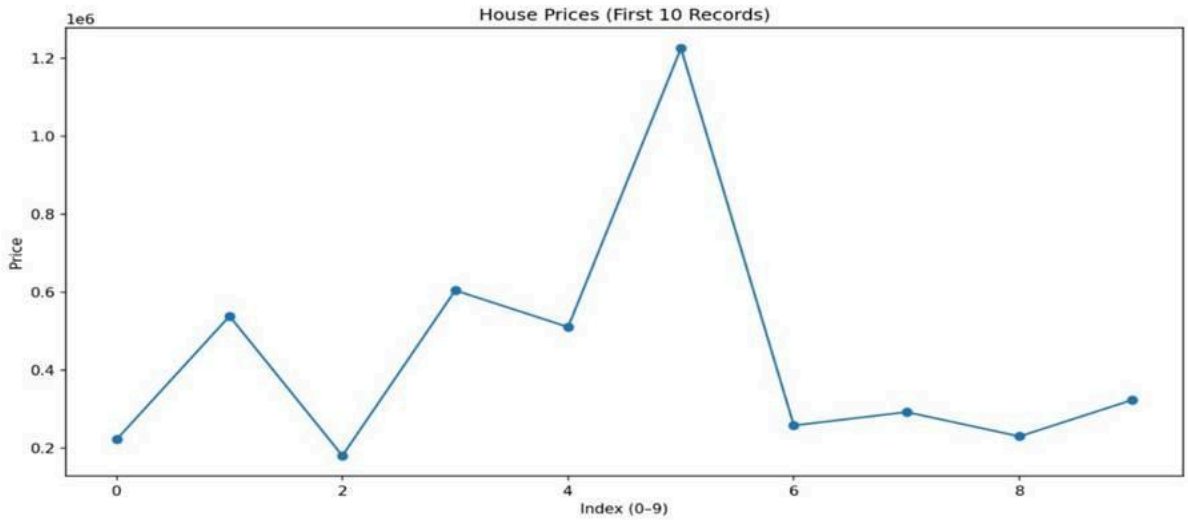
Description:

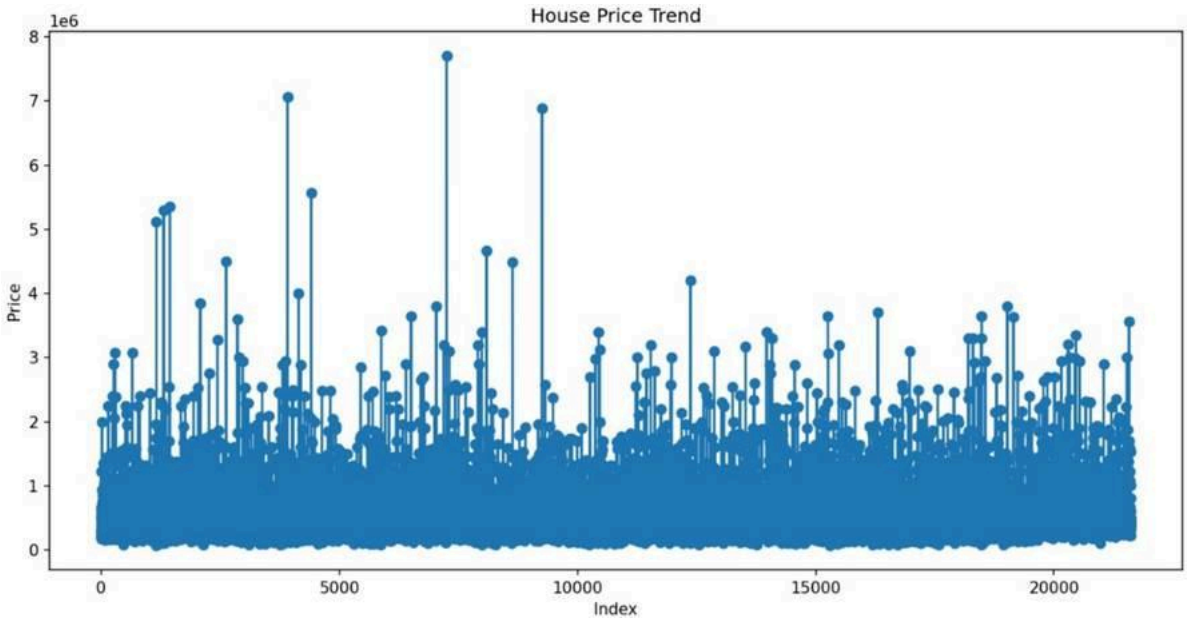
In this scenario,

Line graph visualizes the price fluctuations across the first ten entries of the dataset. By isolating the id and price columns and converting the price values into a NumPy array, the data was prepared for a clean plotting process. The X-axis represents the sequential index of the records, ranging from 0 to 9, while the Y-axis tracks the corresponding house prices. The resulting plot clearly labels the trend with a descriptive title and axis titles, ensuring the visual information is easy to interpret.

Task	Description
1.	Select: id & price
2.	Take the first 10 rows only.
3.	Convert the Price into a NumPy Array
4.	Plot a line graph with X-axis – (index [0-9]), Y-axis – (Height)
5.	Add the title and labels
6.	Save the graph: plt.savefig("house_prices_line.png")

Graph:





Conclusion:

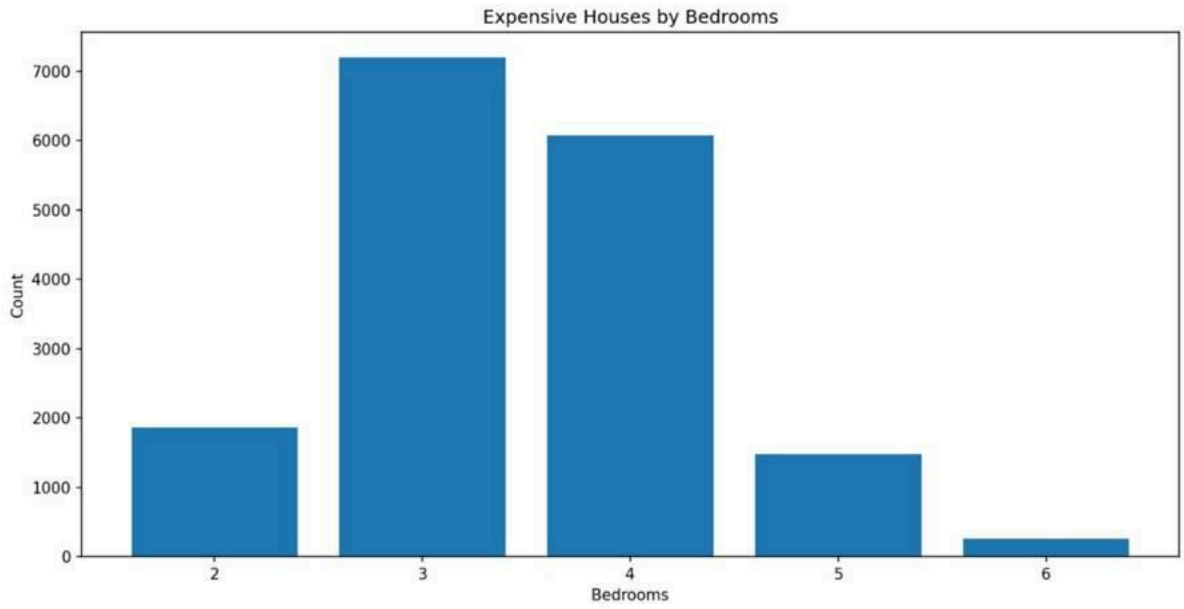
This visualization provides a focused look at the initial pricing trends within the data. By strictly limiting the scope to the first 10 rows, we can identify specific volatility or patterns that might be lost in a larger dataset. The process of converting the data to a NumPy array and using Matplotlib's saving function ensures that the insights are not only viewable but also preserved as a high-quality image file.

Scenario 3: Filtering + Bar Chart (🟡 Medium)

Description:
In this scenario,
This project analyzes the KC House dataset using Python libraries like Pandas and Matplotlib.[Text Wrapping Break] The data is first cleaned by handling missing values and converting columns to proper formats.[Text Wrapping Break] Expensive houses are filtered based on price conditions.[Text Wrapping Break] The data is then grouped by bedrooms to study distribution patterns.[Text Wrapping Break] A bar chart is created and saved to visualize the results.

Task	Description
1.	Filter houses where: ○ price > 1000000
2.	Count number of houses per: ○ bedrooms
3.	Select top bedroom categories.
4.	Convert results to NumPy arrays.
5.	Plot a bar chart: ○ X-axis → Bedrooms ○ Y-axis → Count
6.	Rotate labels if needed.
7.	Save graph: plt.savefig("expensive_houses_bar.png")

Graph:



Conclusion:

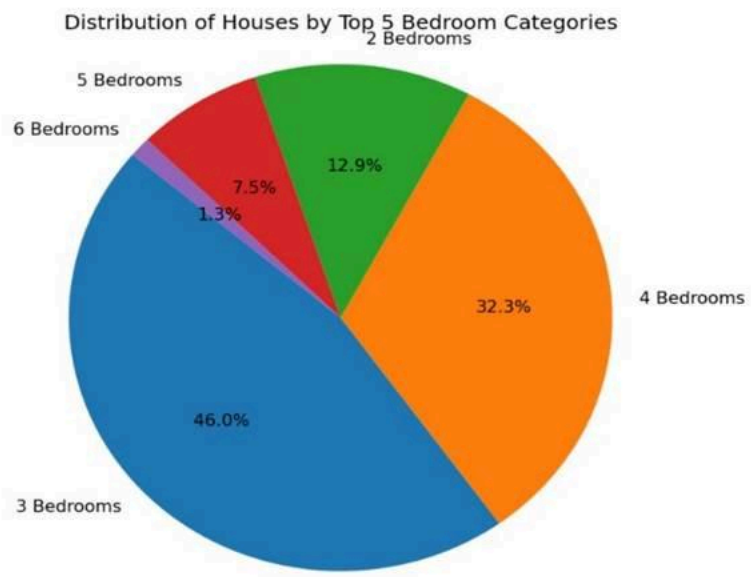
The analysis shows that house prices vary depending on the number of bedrooms in a house. Some bedroom categories appear more frequently among expensive houses, indicating a relationship between size and price. Proper data cleaning is important because it improves the accuracy and reliability of the results. Visualization using bar charts makes it easier to identify patterns and compare different groups clearly. Overall, this project demonstrates how Python can be effectively used for data analysis and decision-making.

Scenario 4: Pie Chart (Region Distribution) + Save (🟡 Medium)

Description:
In this scenario,
The House Sales dataset (kc_house_data.csv) is loaded using the Pandas library to analyze the composition of the housing market.
The number of houses is counted based on the 'bedrooms' column to determine the frequency of each house type.
The top 5 bedrooms categories are selected to ensure the visualization remains clear and focuses on the most significant market segments.
Labels and values are prepared, and a pie chart is generated using Matplotlib to represent the data distribution visually.
Percentage labels are added to each slice using the autopct parameter to show the relative market share of each category.
Finally, the chart is saved as an image file named bedroom_distribution.png for use in reports and presentations.

Task	Description
1.	Count number of houses by: ○ bedrooms
2.	Select top 5 bedrooms categories.
3.	Prepare labels and values.
4.	Plot a pie chart.
5.	Add percentage labels using autopct (e.g., '%.1f%%').
6.	Save the graph: plt.savefig("bedroom_distribution.png")

Graph:



Conclusion:
This scenario helps in understanding the distribution of house types by visualizing the most common bedroom configurations. By filtering for the top 5 categories, the analysis provides a concise overview of the market, highlighting that 3 and 4-bedroom homes make up the majority of the inventory. The use of percentage labels and a pie chart format make the data easy to interpret at a glance, improving the communication of market trends. As a result, this visualization becomes a valuable tool for identifying the standard property types in the region and supporting strategic real estate decisions**.**

Scenario 5: Advanced Analysis + Multiple Graphs (🔴 Hard)

Description:
In this scenario,
It performs advanced analysis on the house dataset using feature engineering, NumPy operations, and multiple visualizations. A new column Price Category is created to classify houses into Luxury, Midrange, and Affordable based on price. The price column is converted into a NumPy array, and np.diff() is used to analyze price variations. A line graph shows overall price trends, a stacked bar chart represents the distribution of price categories across bedrooms, and a histogram illustrates the distribution of house prices. All graphs are saved for reporting purposes.

Task	Description
1.	Feature Creation Create a new column: Price Category ● price >= 1000000 → "Luxury" ● 500000 – 999999 → "Mid Range" ● < 500000 → "Affordable"
2.	1. Convert price column to a NumPy array. 2. Calculate price differences using: np.diff()
3.	Visualizations
4.	Save all graphs plt.savefig("price_trend.png") plt.savefig("price_category_stacked.png") plt.savefig("price_histogram.png")
5.	1. Which bedroom category has the most expensive houses? 2. Which price category is most common? 3. What is the distribution pattern of house prices? ○ Right-skewed? ○ Normal? ○ Concentrated in a lower price range?
7.	Save the graphs: plt.savefig("height_trend.png"), plt.savefig("height_category_stacked.png"), plt.savefig("height_histogram.png").
8.	**Identify**: Which region has tallest hills, Which category is most common, and Distribution pattern of heights.

Result:

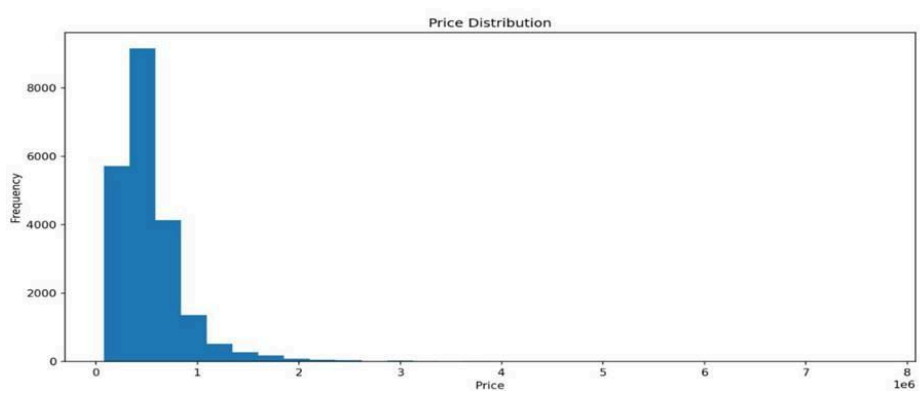
```
Top Bedroom Categories:
bedrooms
3    7206
4    6078
2    1861
5    1482
6     252
Name: count, dtype: int64
Pie chart successfully generated and saved as 'bedroom_distribution.png'

Price Category Counts:
price_category
Affordable    12408
Mid Range      7713
Luxury         1492
Name: count, dtype: int64

First 10 Price Differences:
[ 316100. -358000.  424000.  -94000.  715000. -967500.   34350.  -62350.
   93500.  339500.]

========== INSIGHTS ==========
1. Bedroom category with most expensive houses: 4
2. Most common price category: Affordable
3. Price distribution is right-skewed (most houses are in lower price range with few very high values).
```

Graphs:



Conclusion:

The analysis shows that houses with more bedrooms are more likely to fall under the luxury category. Most houses belong to the affordable or mid-range categories, indicating fewer high-priced properties. The price distribution is right-skewed, meaning most values are concentrated in the lower range with a few extreme high prices. Overall, the dataset reflects a typical real estate pattern with uneven price distribution.

Final Conclusion:

This project helped analyze the KC House Sales dataset using Python libraries such as Pandas, NumPy, and Matplotlib. The dataset was cleaned by handling missing values and converting important columns into proper numeric formats. Different visualizations like line graphs, bar charts, pie charts, stacked bar charts, and histograms were created to understand house prices and bedroom distribution. Expensive houses were filtered to study luxury homes, and price categories were created for deeper analysis. The results showed that house prices are mostly concentrated in lower and mid ranges, while luxury houses are fewer in number.

What have we learned?

We learnt how to load and clean real-world datasets using Pandas. We understood the importance of handling missing values using mean and mode methods and converting columns into proper numeric data types. We practiced filtering data based on specific conditions and grouping data using functions like groupby() and value_counts(). We also learnt how to use NumPy arrays and functions such as np.diff() for analysis. Using Matplotlib, we created different types of graphs and saved them using plt.savefig(). Most importantly, we learnt how to identify patterns in data and draw meaningful conclusions from the analysis.